



Universidade Federal de Santa Catarina – UFSC
DEC – Engenharia de Computação

Linguagem de Descrição de Hardware

Dr. Ney Laert Vilar Calazans

Última alteração: 02/06/2025

❖ Baseado em materiais originais dos Profs. **Ney Calazans**, **Fernando Moraes (PUCRS)**, e **Rafael Garibotti (PUCRS)**

Um Tutorial de PROTOTIPAÇÃO EM FPGAs Usando a Plataforma Nexys A7 e o Software Xilinx VIVADO

USANDO O AMBIENTE VIVADO DA XILINX

➤ Preparando-se para usar o ambiente VIVADO na PUCRS

1. Logar-se no Sistema Operacional Linux (Debian) do laboratório
2. Abrir um janela de terminal (para lançar comandos de texto no *shell*)
3. Executar o comando abaixo para ter acesso às ferramentas de CAD
 - ✓ `Prompt> source /soft64/source_gaph`
4. Executar o comando para configura o uso do ambiente VIVADO
 - ✓ `Prompt> module load vivado`
5. Executar o VIVADO (em background)
 - ✓ `Prompt> vivado &`

UTILIZANDO FPGAs PARA PROTOTIPAÇÃO

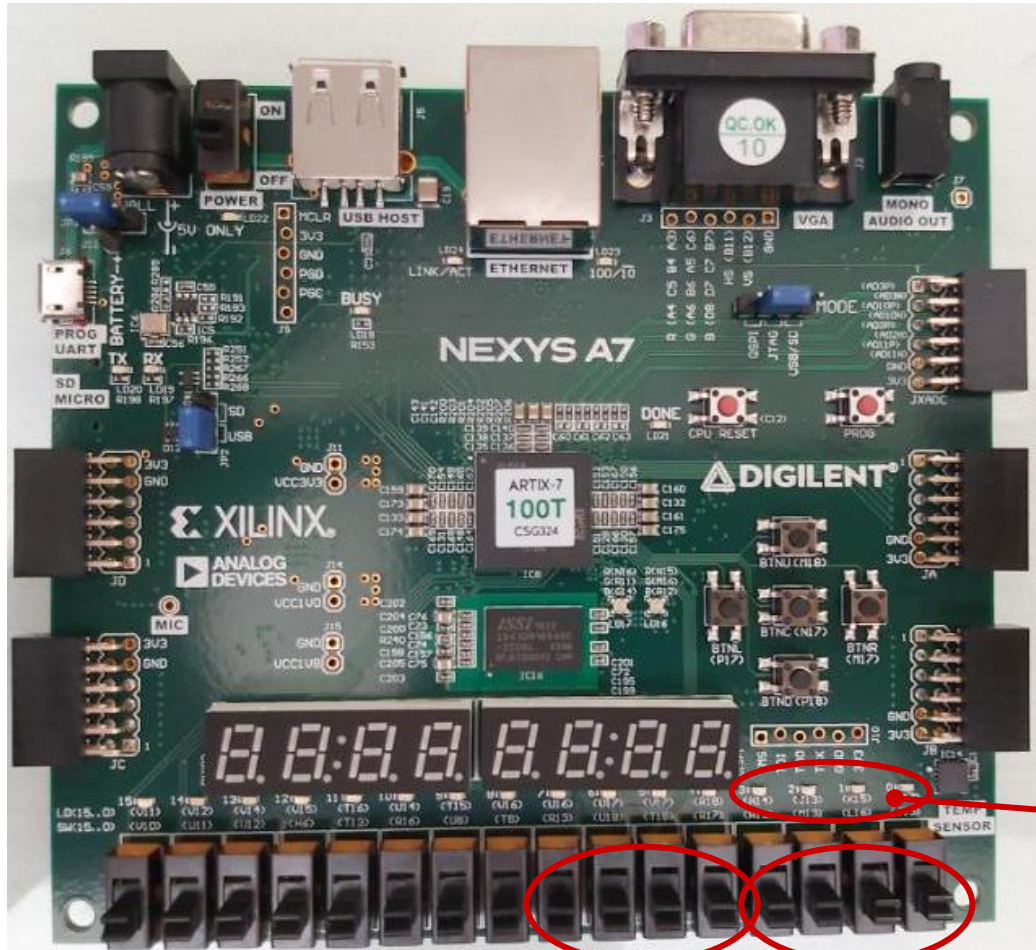
- Inicialmente, garanta que **Vivado está instalado** e que a placa **Nexys A7** esteja conectada ao seu computador pelo **cabo USB desde antes o computador ter sido ligado** (por garantia...). Garanta também que a placa está configurada para **ser alimentada pela USB (Jumper JP3 da placa Nexys A7, detalhes no manual de referência da placa)**, configurada para **receber bitstreams via JTAG (JP1)** e ligada
- 1. Abaixo aparece um circuito somador, um dos usados no trabalho T1, que será prototipado nesta aula (**compare com a sua implementação do Projeto2 daquele trabalho**). Abra o Vivado e crie um projeto de nome **somador4** com este arquivo como único fonte sintetizável, conforme visto no tutorial do T1, sem incluir um **testbench**

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all; --Para permitir soma de std_logic

entity somador is
port (A, B: in std_logic_vector(3 downto 0);
      Soma: out std_logic_vector(3 downto 0));
end somador;

architecture somador of somador is
begin
    Soma <= A + B; --Soma de dois vetores de 4 bits
end somador;
```

ONDE AS ENTRADAS E SAÍDAS SE CONECTAM?



B(3 downto 0) A(3 downto 0)

- Placas de prototipação têm recursos de entrada e saída conectados ao FPGA
 - ✓ LEDs, chaves, displays, chaves, interfaces serial, USB, Ethernet, vídeo, etc...
- ✓ Ao lado aparece um exemplo de escolha de dispositivos de entrada e saída da placa Nexys A7 para prototipar o somador de 4 bits deste trabalho
- ✓ São 4 chaves (SW7 a SW4) para a entrada B, 4 chaves (SW3 a SW0) para a entrada A e 4 leds (LD3 a LD0) para mostrar o resultado da soma

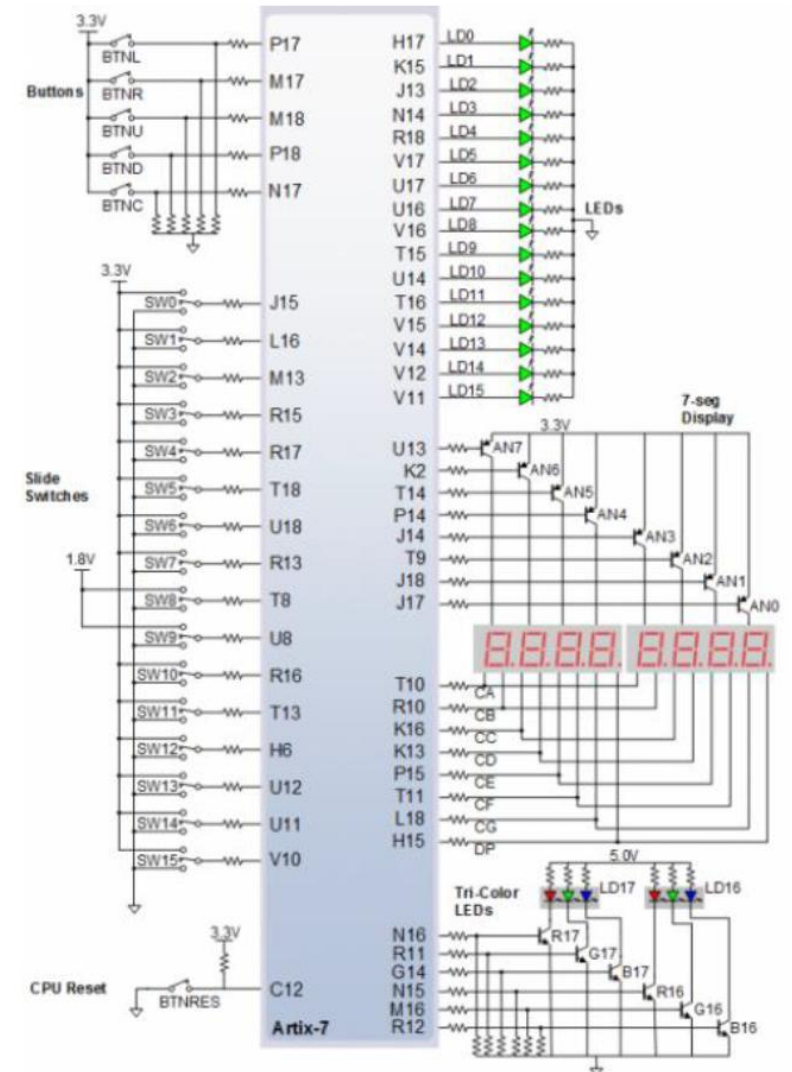
Soma(3 downto 0)

ONDE AS ENTRADAS E SAÍDAS SE CONECTAM?

2. Abra o manual de referência da placa que está disponível via página de apoio com link disponível no Moodle ou [aqui](#). O Manual de referência é o arquivo de nome [nexys-a7_rm.pdf](#). Ir na página 22 do manual em pauta e achar a Figura 9.1, reproduzida à direita e estude este material

- ✓ Um arquivo do projeto VIVADO deve relacionar as entradas e saídas do hardware descrito em VHDL com os pinos do FPGA conectados aos recursos da placa que se quer usar
- ✓ Este arquivo se chama **Xilinx Design Constraints (Restrições de Projeto da Xilinx)**. No Vivado, ele é um arquivo que usa a extensão **.xdc** por omissão

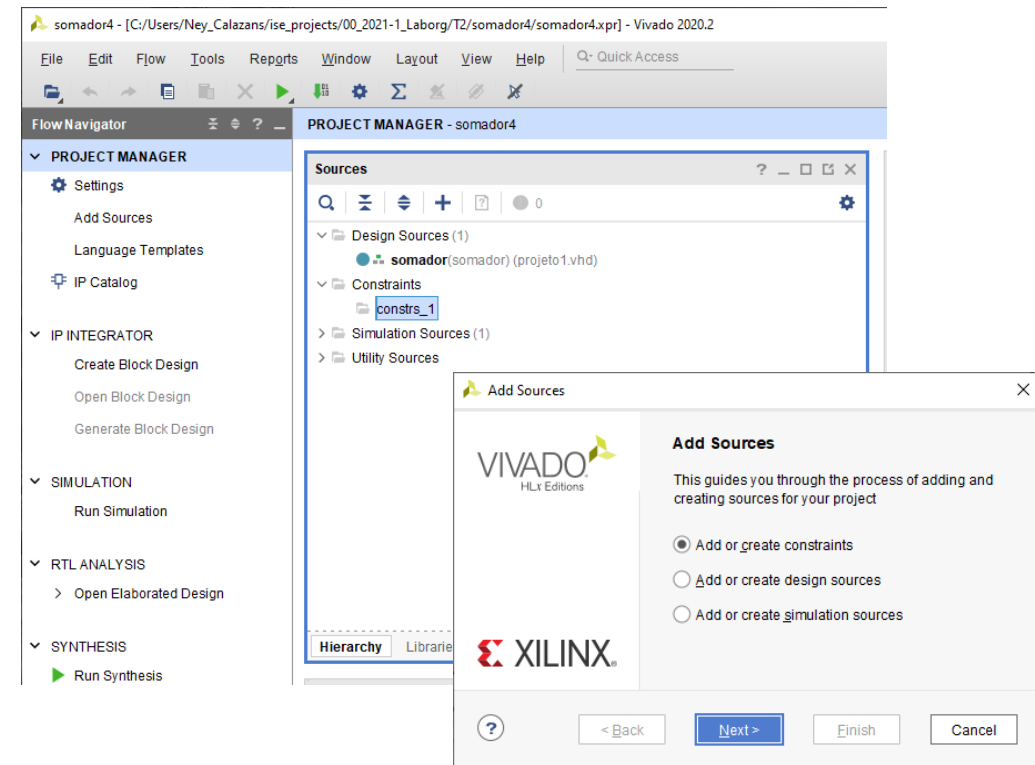
Nexys A7 – E/S Básica



MONTANDO O PROJETO DE PROTOTIPAÇÃO

Criação do arquivo **.xdc** – Este arquivo define a relação entre cada nome de pino da entidade VHDL de mais alto nível do projeto de vocês e os pinos físicos do FPGA

3. Para criar este arquivo, vá na página de material de apoio da placa [aqui](#). Baixe o arquivo geral **Nexys-A7-100T-Master.txt**. Depois de baixar, renomeie ele para **Nexys-A7-100T-Master.xdc**. Este contém todos os pinos da placa Nexys A7 em linhas comentadas
4. Adicione este arquivo no projeto que você criou, da seguinte maneira: (1) na janela **Sources** do **Project Manager** selecione o folder **constrs_1** dentro do folder **Constraints**; (2) Clique com o botão direito do mouse no folder selecionado e escolha **+ Add Sources**. A janela de diálogo deve ser como ao lado. Escolha a opção **Add or create constraints** e clique em **Next**. Na janela seguinte, escolha **Add Files**, navegue ao local onde você baixou o arquivo geral **.xdc** e escolha ele; Em seguida clique em **Finish**



MONTANDO O PROJETO DE PROTOTIPAÇÃO

- Abra no editor de texto do Vivado o arquivo **.xdc** e edite ele de acordo com o que está dito nos comentários das 4 primeiras linhas deste; não esqueça de salvar o arquivo depois da edição
- Use o diagrama de pinagem **Nexys A7 – E/S Básica** mostrado em slide anterior desta apresentação e a escolha de chaves e leds também mostrada em slide anterior; o resultado esperado aparece ao lado

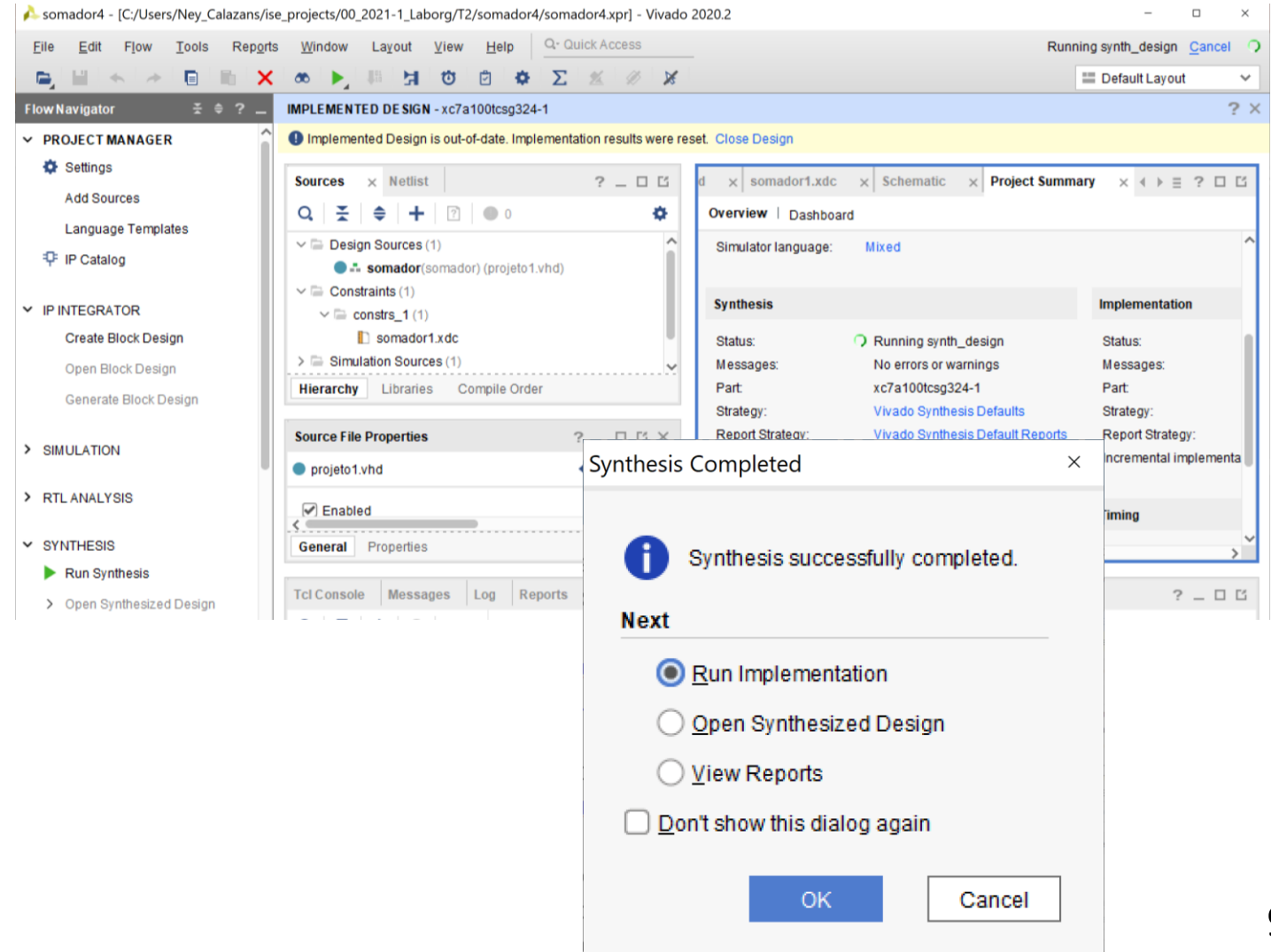
The screenshot displays the Vivado IDE interface. On the left, the 'Project Manager' window shows the project hierarchy for 'somador4'. Under 'Design Sources', 'somador(somador) (projeto1.vhd)' is listed. Under 'Constraints', 'constrs_1(1)' contains 'somador1.xdc'. The 'Source File Properties' window for 'somador1.xdc' is open, showing it is an 'XDC' file, 19.3 KB, and located at 'C:/Users/Ney_Calazans/ise_projects/00_2021-1_Laborg/T2/somador4/somador4.srsrcs/constrs_1/imports/new/somador1.xdc'. The main editor window shows the contents of 'somador1.xdc', which is a Tcl script for configuring the Nexys A7-100T. The script includes comments and commands for setting properties, creating a clock, and configuring switches and LEDs. The script is as follows:

```
1 # Save File (Ctrl+S) general .xdc for the Nexys A7-100T
2 ##
3 ## - uncomment the lines corresponding to used pins
4 ## - rename the used ports (in each line, after get_ports) according to the top level signal names in the project
5
6 ## Clock signal
7 #set_property -dict { PACKAGE_PIN E3 IOSTANDARD LVCMOS33 } [get_ports { CLK100MHz }]; #IO_L12P_T1_MRCC_35 Sch=clk100mhz
8 #create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {CLK100MHz}];
9
10
11 ##Switches
12 set_property -dict { PACKAGE_PIN J15 IOSTANDARD LVCMOS33 } [get_ports { A[0] }]; #IO_L24N_T3_R50_15 Sch=sw[0]
13 set_property -dict { PACKAGE_PIN L16 IOSTANDARD LVCMOS33 } [get_ports { A[1] }]; #IO_L3N_T0_DQS_EMCCLK_14 Sch=sw[1]
14 set_property -dict { PACKAGE_PIN M13 IOSTANDARD LVCMOS33 } [get_ports { A[2] }]; #IO_L6N_T0_D08_VREF_14 Sch=sw[2]
15 set_property -dict { PACKAGE_PIN R15 IOSTANDARD LVCMOS33 } [get_ports { A[3] }]; #IO_L13N_T2_MRCC_14 Sch=sw[3]
16 set_property -dict { PACKAGE_PIN R17 IOSTANDARD LVCMOS33 } [get_ports { B[0] }]; #IO_L12N_T1_MRCC_14 Sch=sw[4]
17 set_property -dict { PACKAGE_PIN T18 IOSTANDARD LVCMOS33 } [get_ports { B[1] }]; #IO_L7N_T1_D10_14 Sch=sw[5]
18 set_property -dict { PACKAGE_PIN U10 IOSTANDARD LVCMOS33 } [get_ports { B[2] }]; #IO_L17N_T2_A13_D29_14 Sch=sw[6]
19 set_property -dict { PACKAGE_PIN R13 IOSTANDARD LVCMOS33 } [get_ports { B[3] }]; #IO_L6N_T0_D07_14 Sch=sw[7]
20 #set_property -dict { PACKAGE_PIN T8 IOSTANDARD LVCMOS18 } [get_ports { SW[8] }]; #IO_L24N_T3_34 Sch=sw[8]
21 #set_property -dict { PACKAGE_PIN U8 IOSTANDARD LVCMOS18 } [get_ports { SW[9] }]; #IO_25_34 Sch=sw[9]
22 #set_property -dict { PACKAGE_PIN R16 IOSTANDARD LVCMOS33 } [get_ports { SW[10] }]; #IO_L15P_T2_DQS_RDR_B_14 Sch=sw[10]
23 #set_property -dict { PACKAGE_PIN T13 IOSTANDARD LVCMOS33 } [get_ports { SW[11] }]; #IO_L23P_T3_A03_D19_14 Sch=sw[11]
24 #set_property -dict { PACKAGE_PIN H6 IOSTANDARD LVCMOS33 } [get_ports { SW[12] }]; #IO_L24P_T3_35 Sch=sw[12]
25 #set_property -dict { PACKAGE_PIN U12 IOSTANDARD LVCMOS33 } [get_ports { SW[13] }]; #IO_L20P_T3_A08_D24_14 Sch=sw[13]
26 #set_property -dict { PACKAGE_PIN U11 IOSTANDARD LVCMOS33 } [get_ports { SW[14] }]; #IO_L19N_T3_A09_D25_VREF_14 Sch=sw[14]
27 #set_property -dict { PACKAGE_PIN V10 IOSTANDARD LVCMOS33 } [get_ports { SW[15] }]; #IO_L21P_T3_DQS_14 Sch=sw[15]
28
29 ## LEDs
30 set_property -dict { PACKAGE_PIN H17 IOSTANDARD LVCMOS33 } [get_ports { Soma[0] }]; #IO_L18P_T2_A24_15 Sch=led[0]
31 set_property -dict { PACKAGE_PIN K15 IOSTANDARD LVCMOS33 } [get_ports { Soma[1] }]; #IO_L24P_T3_RS1_15 Sch=led[1]
32 set_property -dict { PACKAGE_PIN J13 IOSTANDARD LVCMOS33 } [get_ports { Soma[2] }]; #IO_L17N_T2_A26_15 Sch=led[2]
33 set_property -dict { PACKAGE_PIN N14 IOSTANDARD LVCMOS33 } [get_ports { Soma[3] }]; #IO_L8P_T1_D11_14 Sch=led[3]
34 #set_property -dict { PACKAGE_PIN R18 IOSTANDARD LVCMOS33 } [get_ports { LED[4] }]; #IO_L7P_T1_D09_14 Sch=led[4]
35 #set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports { LED[5] }]; #IO_L18N_T2_A11_D27_14 Sch=led[5]
36 #set_property -dict { PACKAGE_PIN U17 IOSTANDARD LVCMOS33 } [get_ports { LED[6] }]; #IO_L17P_T2_A14_D30_14 Sch=led[6]
37 #set_property -dict { PACKAGE_PIN U16 IOSTANDARD LVCMOS33 } [get_ports { LED[7] }]; #IO_L18P_T2_A12_D28_14 Sch=led[7]
38 #set_property -dict { PACKAGE_PIN V16 IOSTANDARD LVCMOS33 } [get_ports { LED[8] }]; #IO_L16N_T2_A15_D31_14 Sch=led[8]
39 #set_property -dict { PACKAGE_PIN T15 IOSTANDARD LVCMOS33 } [get_ports { LED[9] }]; #IO_L14N_T2_SRCC_14 Sch=led[9]
40 #set_property -dict { PACKAGE_PIN U14 IOSTANDARD LVCMOS33 } [get_ports { LED[10] }]; #IO_L22P_T3_A05_D21_14 Sch=led[10]
41 #set_property -dict { PACKAGE_PIN T16 IOSTANDARD LVCMOS33 } [get_ports { LED[11] }]; #IO_L15N_T2_DQS_DOUT_C50_B_14 Sch=led[11]
42 #set_property -dict { PACKAGE_PIN V15 IOSTANDARD LVCMOS33 } [get_ports { LED[12] }]; #IO_L16P_T2_CSI_B_14 Sch=led[12]
43 #set_property -dict { PACKAGE_PIN V14 IOSTANDARD LVCMOS33 } [get_ports { LED[13] }]; #IO_L22N_T3_A04_D20_14 Sch=led[13]
44 #set_property -dict { PACKAGE_PIN V12 IOSTANDARD LVCMOS33 } [get_ports { LED[14] }]; #IO_L20N_T3_A07_D23_14 Sch=led[14]
45 #set_property -dict { PACKAGE_PIN V11 IOSTANDARD LVCMOS33 } [get_ports { LED[15] }]; #IO_L21N_T3_DQS_A06_D22_14 Sch=led[15]
```

PROCESSANDO O PROJETO DE PROTOTIPAÇÃO

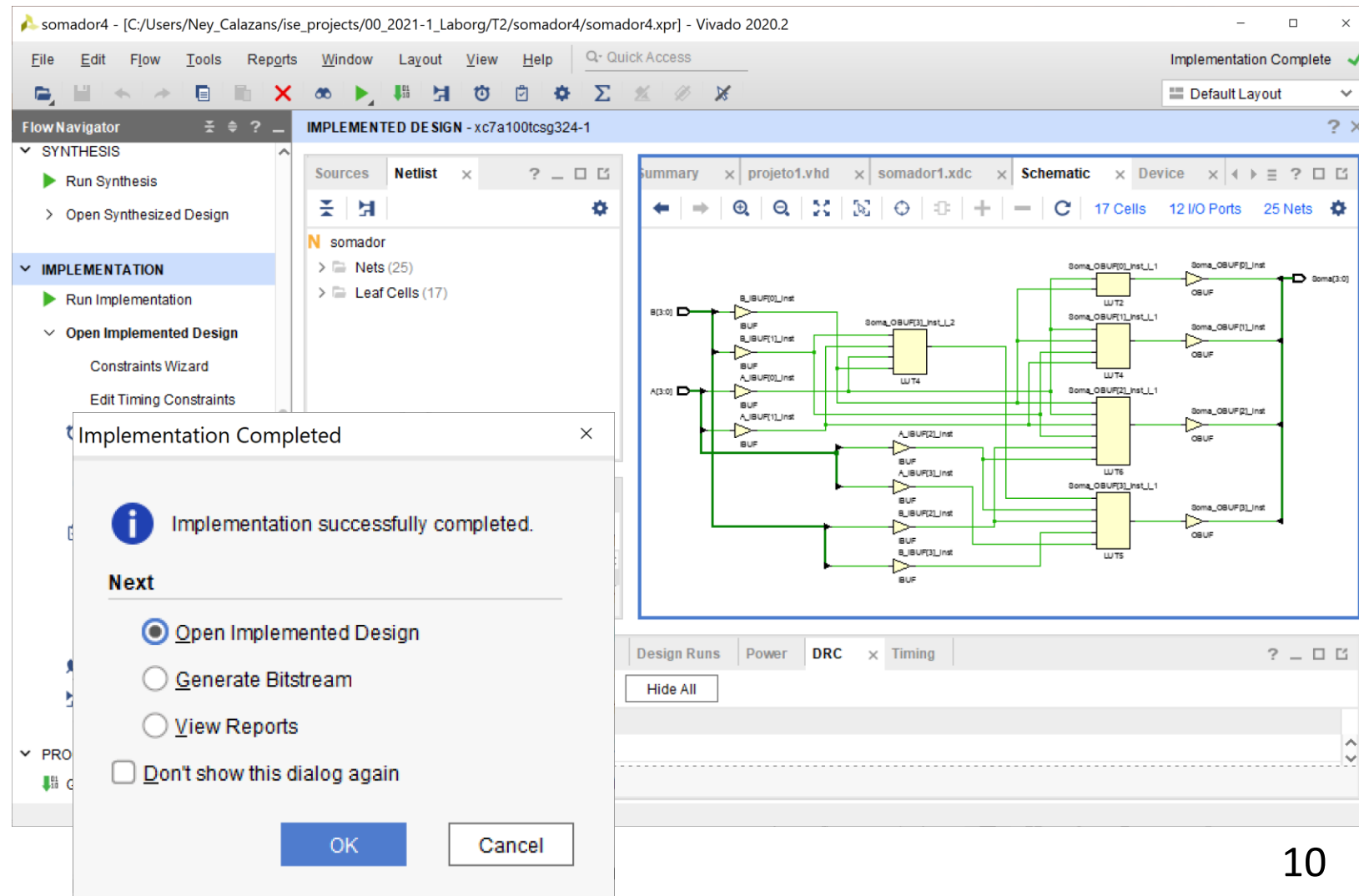
Após o passo 6, o projeto está pronto para ser processado e gerar o protótipo de hardware, usando, por exemplo, os passos seguintes

7. No quadro **Flow Navigator**, feche as abas **SIMULATION** e **RTL ANALYSIS**, pois não as usaremos aqui;
8. Selecione a entidade/arquitetura (E/A) somador no quadro **Sources**, folder **Design Sources** como ao lado
9. No **Flow Navigator**, clique em **Run Synthesis** e veja a evolução da síntese na aba **Project Summary** do quadro **ELABORATED DESIGN**
10. Ao final da síntese, aceite a sugestão da janela que surge (**Run Implementation**) e realize a implementação do hardware clicando **OK** nesta janela; acompanhe o processo no mesmo local de antes (**Project Summary**)



PROCESSANDO O PROJETO DE PROTOTIPAÇÃO

- Após o passo 10, o projeto pode ser visualizado em diferentes formas usando, por exemplo, os passos a seguir
11. Ao final da implementação, aceite a sugestão da janela que surge (**Open Implemented Design**) clicando **OK**
 12. Na janela superior direita do quadro **IMPLEMENTED DESIGN** surgem duas novas abas (**Schematic** e **Device**). Abra a aba **Schematic**
 13. Nesta aba veja no topo as estatísticas (**17 Células, 12 Portas de E/S, 25 Redes**). Células são as figuras com fundo **amarelo**, Portas são os pinos hardware e Redes são fios (linhas **verdes**). Linhas **verdes** espessas são conjuntos de fios, linhas **verdes** finas são fios individuais

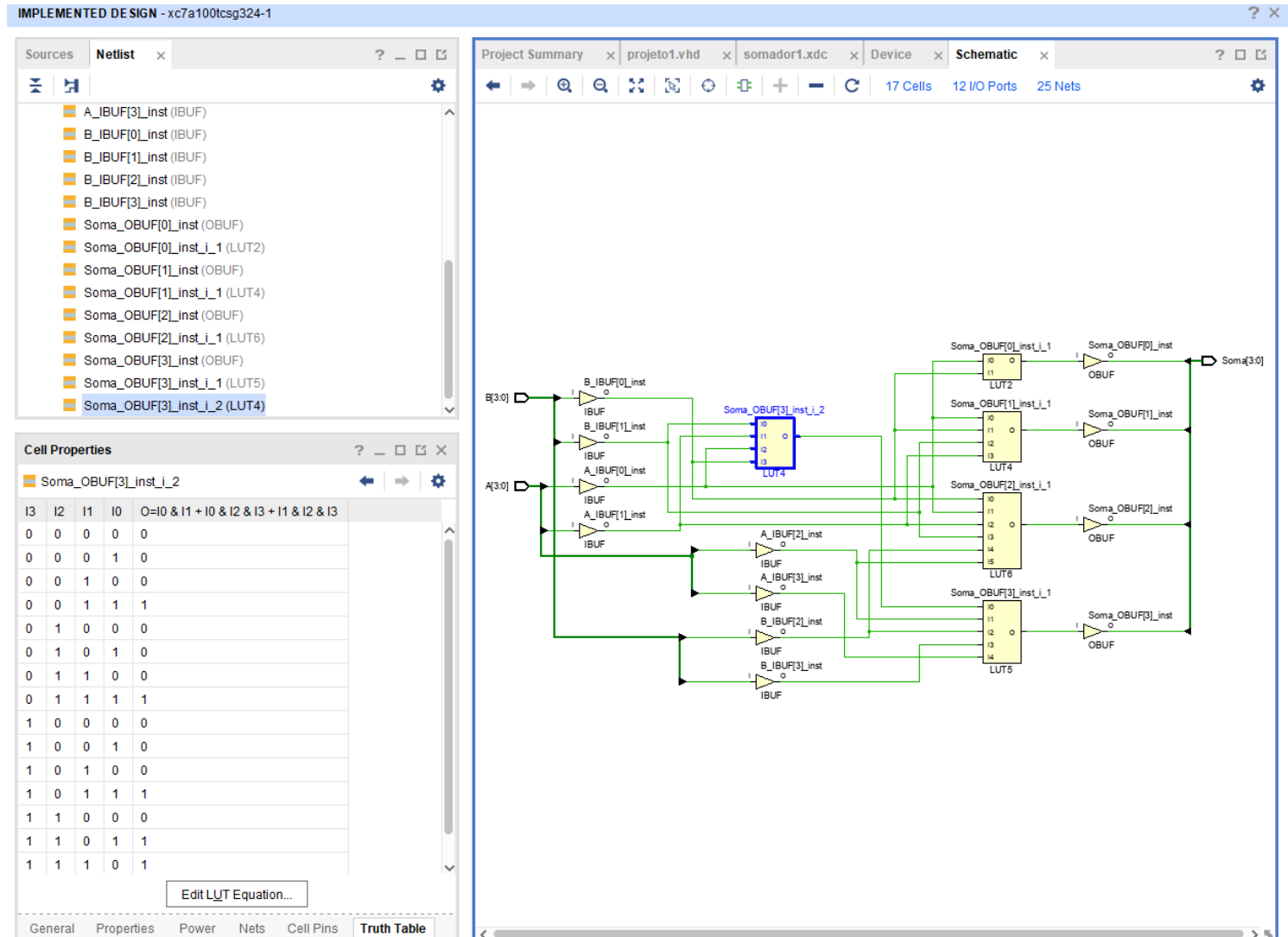


ANALISANDO O PROJETO DE PROTOTIPAÇÃO

Após o passo 13, o projeto pode ser analisado em todos seus detalhes usando, por exemplo, os passos a seguir.

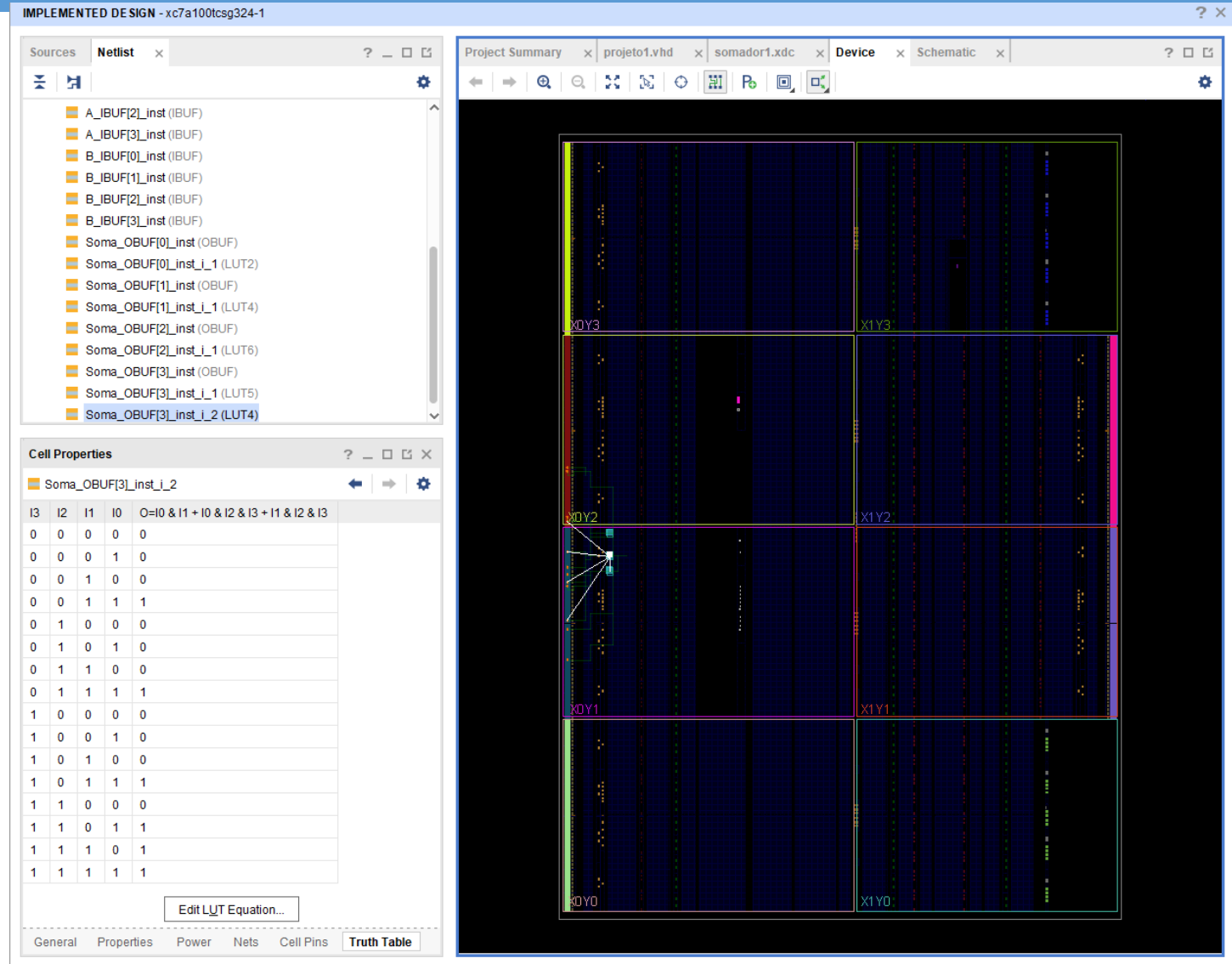
Existem muitas outras possibilidades

14. Confira os números dados, contando células, portas e fios; **identifique os módulos de hardware**, o que eles são?
15. Clique em módulos aleatoriamente e veja o efeito na aba **Netlist** da janela superior esquerda do quadro **IMPLEMENTED DESIGN** e na janela **Cell Properties** deste mesmo quadro; no exemplo, clicamos em uma LUT4 e podemos ver sua tabela verdade e sua equação Booleana; experimente...



ANALISANDO O PROJETO DE PROTOTIPAÇÃO

16. Com a LUT4 do slide anterior selecionada, troque a seleção na janela superior direita do quadro **IMPLEMENTED DESIGN**, da aba **Schematic** para a aba **Device**
17. Veja ao lado a imagem; O retângulo maior dividido em 8 retângulos coloridos representa fisicamente o FPGA inteiro. O quadrado branco é a LUT4 selecionada no esquemático, os fios **brancos** conectados a eles são os caminhos das entradas e da saída da LUT4 para outros componentes; os fios mais longos vêm dos IBUFs (buffers de entrada), localizados junto a pinos de entrada do FPGA e o fio menor conecta a saída da LUT4 a uma entrada da LUT5 (confira no esquemático). Experimente...

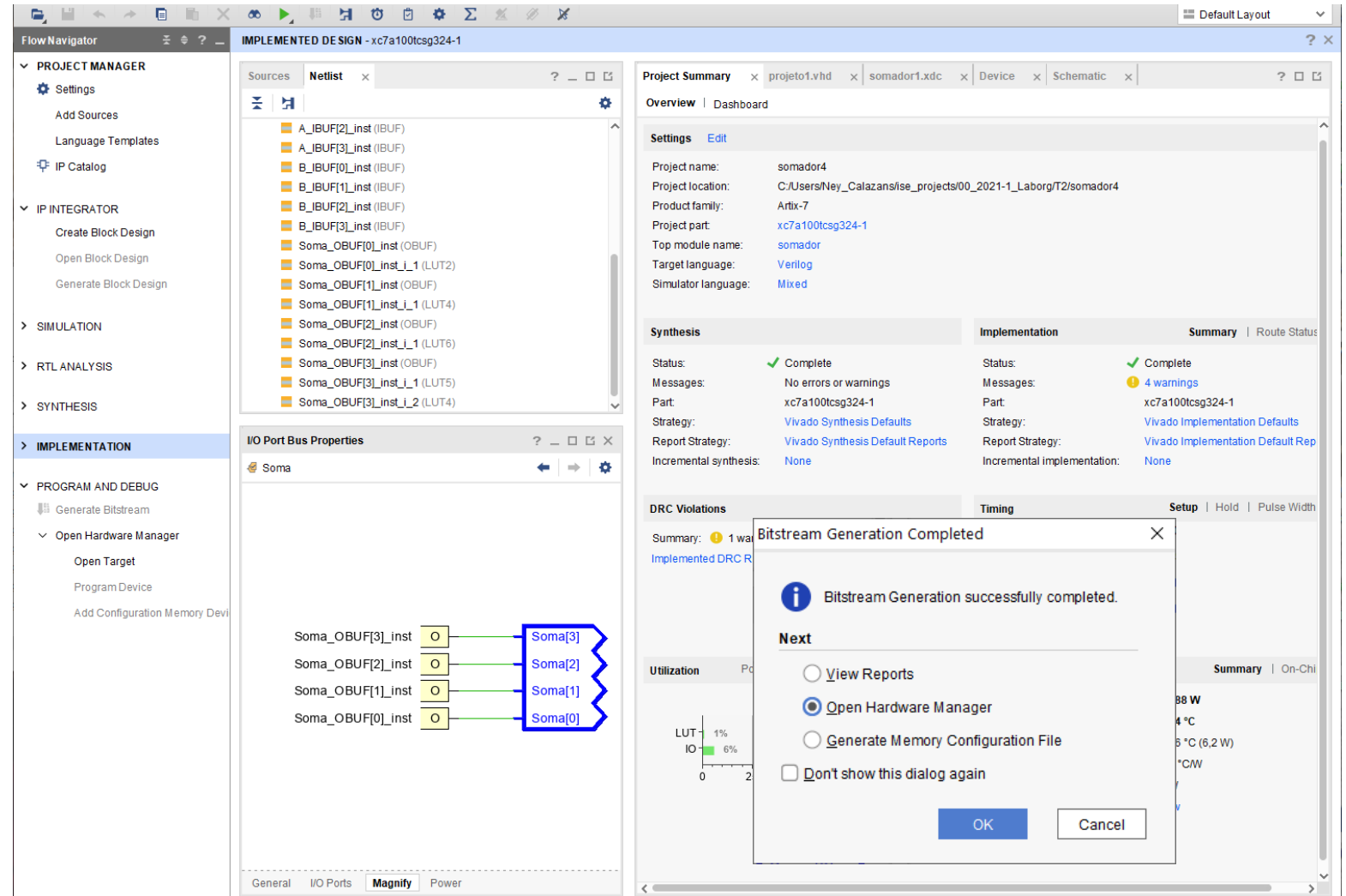


CONCLUINDO O PROJETO DE PROTOTIPAÇÃO

Desde o passo 10, o projeto está implementado, mas falta um passo para poder configurar o hardware da Nexys A7 com ele. *Prototipar o hardware* consiste em tomar o resultado da implementação e gerar o conteúdo necessário para preencher a memória de controle do FPGA com 0s e 1s para criar os componentes e fios exatamente como descrito no circuito. Trata-se de um arquivo binário, enviado para o FPGA via um cabo que conecta-se à placa

18. Na Janela **Flow Navigator**, clique na opção **Generate Bitstream** do menu **PROGRAM AND DEBUG**

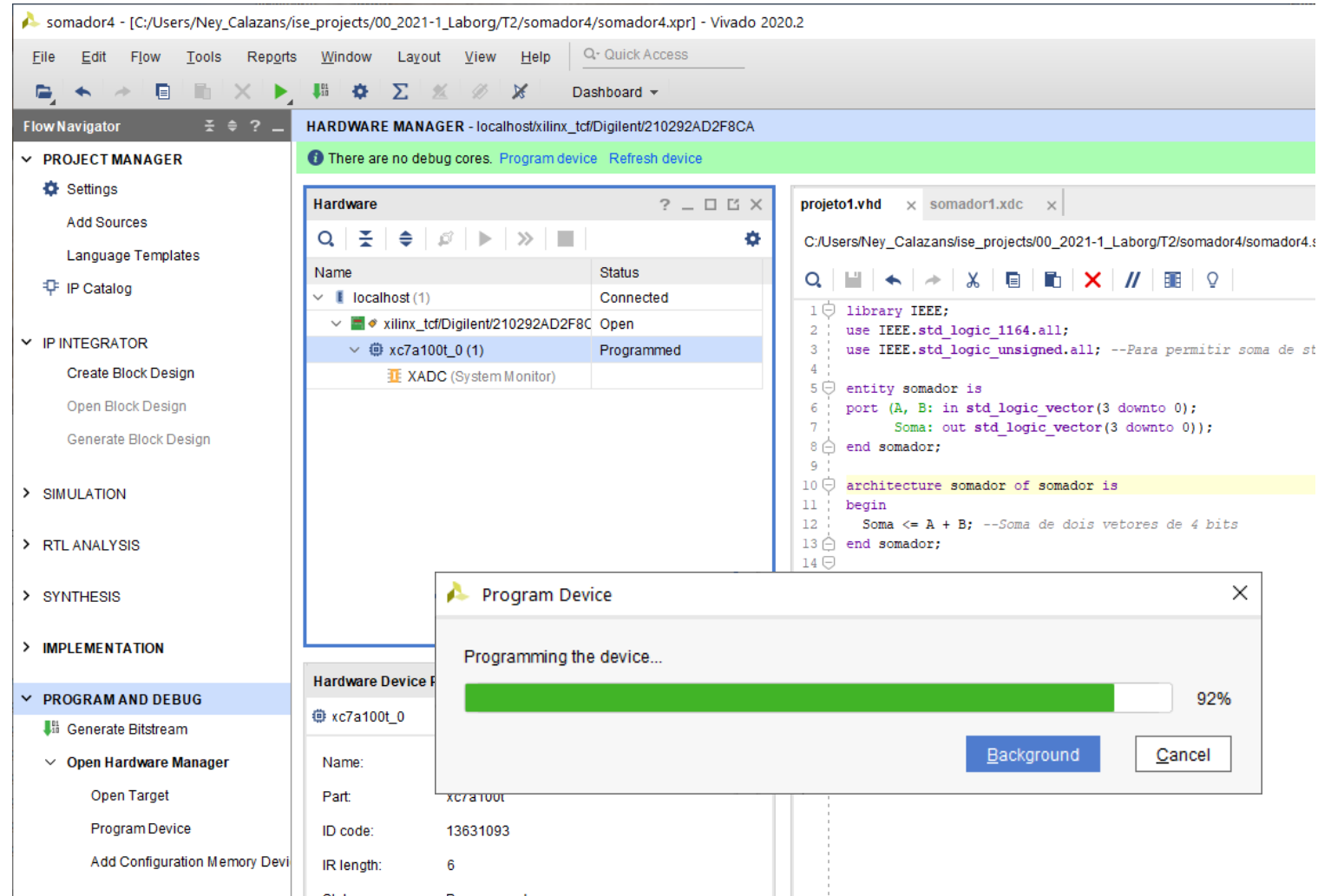
19. Depois de concluída a geração do *bitstream*, escolha a opção **Open Hardware Manager** na janela de *pop-up* que surge



PROTOTIPANDO O PROJETO

Depois do passo 19, o projeto está totalmente pronto e o arquivo de configuração do FPGA está disponível para ser carregado no dispositivo FPGA da placa

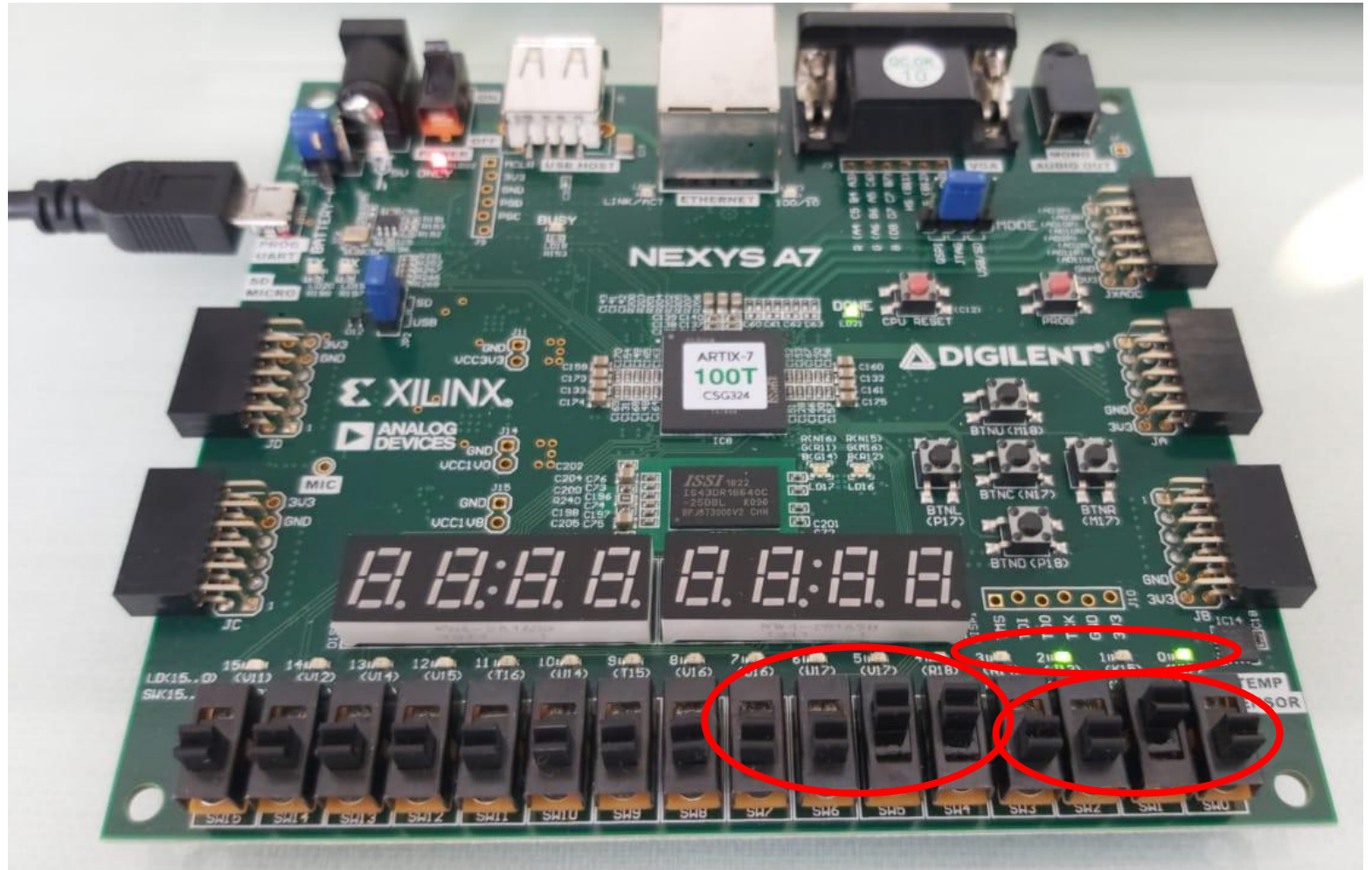
20. Confira que na janela superior esquerda do **HARDWARE MANAGER**, denominada **Hardware**, aparece disponível o dispositivo da plataforma Nexys A7, o **xc7a100t_0**; selecione ele
21. Na Janela **Flow Navigator**, clique na opção **Program Device** do menu **PROGRAM AND DEBUG**



TESTANDO O PROJETO

Depois do passo 21, o projeto está pronto para ser testado na placa

22. Confira que seu projeto funciona. Cuidados: revise que foram respeitadas as instruções prévias fornecidas no primeiro slide deste tutorial (item antes do passo 1)
23. Veja ao lado um exemplo de operação correta do hardware somador de 4 bits: **A(0010) + B(0011) = SOMA (0101)**. Em decimal, $2+3=5$. **A** corresponde às 4 chaves **SW3 a SW0**, **B** às 4 chaves **SW7 a SW4** e **SOMA** aos leds **LD3 a LD0**. As convenções são: Chave para baixo é 0 e chave para cima é 1. **LED** desligado é 0, e **LED** ligado é 1



CONFIGURANDO O FPGA

Alternativa Labs PUCRS

19. A síntese termina por gerar um arquivo com extensão **.bit** que pode ser usado para configurar o hardware no FPGA. Para tanto, usa-se o comando *shell* do Linux **djtgcfg** (*Digilent JTAG Configuration Utility*), instalado nas máquinas do Laboratório e no LAPRO. Executem `djtgcfg ?` para ver as opções deste comando
20. Conectem a placa ao computador via o cabo USB e digitem `djtgcfg enum`. Este comando deve comunicar-se com a placa para identificá-la e obter seus dados, que são impressos na tela
21. Agora digitem `djtgcfg -d NexysA7 init`. Este comando deve comunicar-se com a placa Nexys A7 e listar os dispositivos Xilinx da mesma (o FPGA e a memória flash Xilinx específica, que pode guardar uma configuração completa do FPGA)
22. Finalmente, para configurar o FPGA usem o seguinte comando: síntese: `djtgcfg prog -d NexysA7 -i 0 -f somador4.bit`, **certificando-se de estar no diretório onde se encontra o arquivo gerado pela síntese**
23. O LED amarelo de configuração carregada deve acender e talvez alguns LEDs de dados. Faça testes com o projeto, certificando-se que o somador faz somas de forma correta!

TRABALHO T2

PROJETO 1

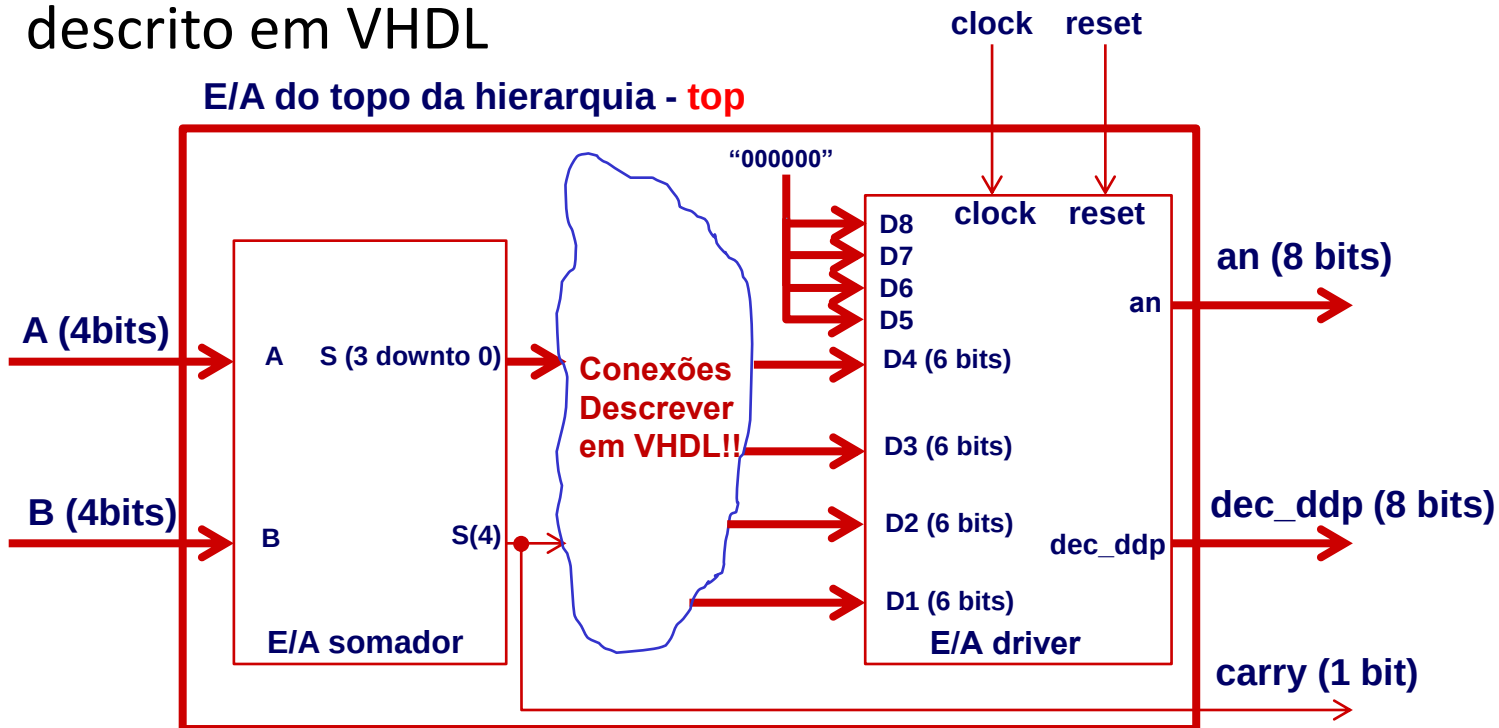
- **Projeto 1** - consiste simplesmente em acrescentar o cálculo de “vai-um” no circuito do somador apresentado no “Tutorial de PROTOTIPAÇÃO EM FPGAs Usando a Plataforma Nexys A7 e o Software Xilinx VIVADO” deste conjunto de slides. Segue uma sugestão de modificações no código para resolver o problema deste projeto
 - ✓ Modificar a saída “Soma” para ser um vetor de 5 bits
 - ✓ Declarar dois sinais internos ‘AA’ e ‘BB’, ambos de 5 bits
 - ✓ Fazer a “Soma <= AA + BB”, e criar ‘AA’ e ‘BB’ através de uma concatenação com ‘0’, e.g.
`AA <= '0' & A; -- o símbolo & significa concatenação em VHDL`
- Modificar o arquivo `.xdc` para que um dos LEDs não usados (por exemplo o LD7) seja associado ao “vai-um” gerado (o quinto bit da soma, designado em VHDL por `SOMA ( 4 )`)
- Simular o VHDL com o mesmo *testbench* do **Projeto 2** do **T1**, para este novo VHDL
- Aplicar os passos descritos na Seção “Tutorial de PROTOTIPAÇÃO EM FPGAs ” ao novo VHDL gerado, prototipando o novo hardware na placa e o testando

PROJETO 2

- **Projeto 2** – Partindo do **Projeto 1**, fazer o resultado da soma aparecer nos mostradores (*displays*) de 7 segmentos em hexadecimal e não em binário nos LEDs. Segue uma sugestão de modificações para resolver o problema. Suponha que o resultado da soma vá para o mostrador mais à direita de um dos grupos de 4 mostradores da placa, e que o “vai-um” seja associado a algum *LED* (pode ser qualquer *LED* não usado, entre os *LEDs* LD4 a LD15 ou um ponto decimal de um dos mostradores de 7 segmentos)
- ✓ Leia o manual de referência [aqui](#) para entender como funcionam os mostradores de sete segmentos da placa Nexys A7 (Seção 9.1 do manual, nas páginas 23 e 24 deste)
 - ✓ A placa Nexys A7 usa dois conjuntos de 4 mostradores de 7 segmentos (**escolha um dos conjuntos para usar neste projeto**). Cada um destes é **multiplexado(?)** e um *hardware* especial precisa ser empregado para viabilizar seu uso, hardware este denominado **acionador** ou **driver**
 - ✓ Abra e estude os comentários iniciais e o VHDL do arquivo disponível em [dspl_drv NexysA7.vhd](#). Este é um exemplo de *driver* para os mostradores.
 - No ambiente VIVADO, crie um novo projeto, adicione os fontes do **Projeto 1** anterior e acrescente o arquivo com o *driver* dado ([dspl_drv NexysA7.vhd](#)) para os mostradores
 - ✓ Modifique o arquivo **.xdc** para adaptá-lo ao novo formato da saída e prototipe o novo circuito

PROJETO 2

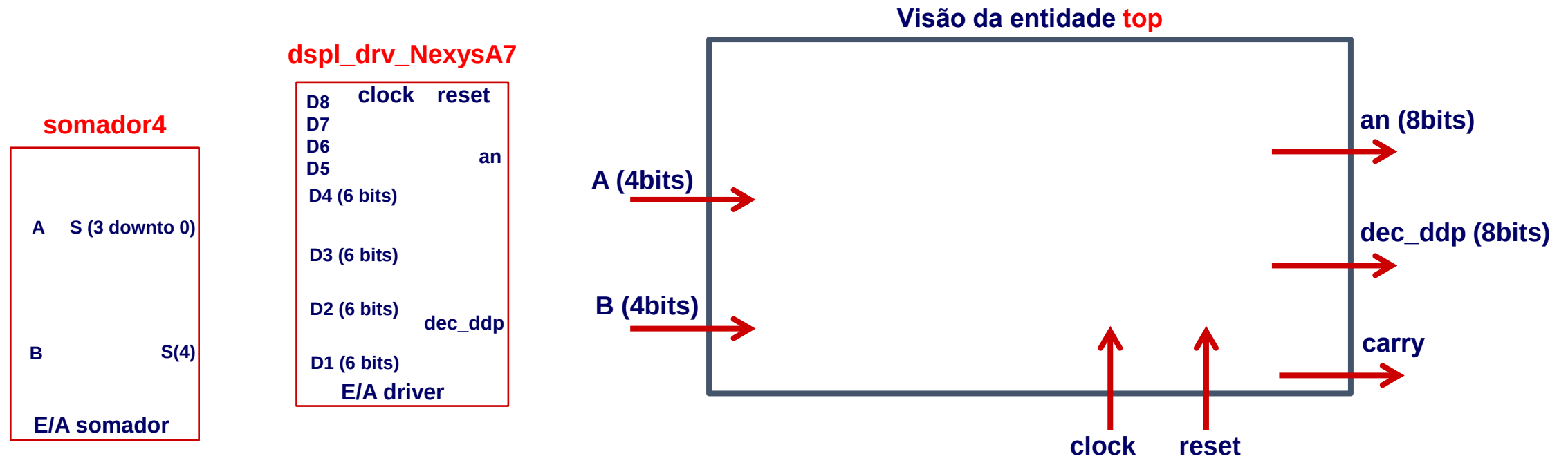
- Para ajudar na compreensão desta parte do trabalho, segue o desenvolvimento de um diagrama parcial de blocos e conexões do circuito resultante. A expressão **E/A** corresponde a um par **Entidade/Arquitetura**, que define um módulo de hardware descrito em VHDL
- clock reset ✓ Trata-se de uma descrição



- ✓ Trata-se de uma descrição hierárquica: O somador e o *driver* são *instanciados* dentro da descrição VHDL da entidade topo da hierarquia de projeto (usando comandos VHDL de instanciação: **port map**)
- ✓ Nem todos os pinos e fios internos estão nomeados explicitamente no diagrama. Complete-os no(s) arquivo(s) VHDL

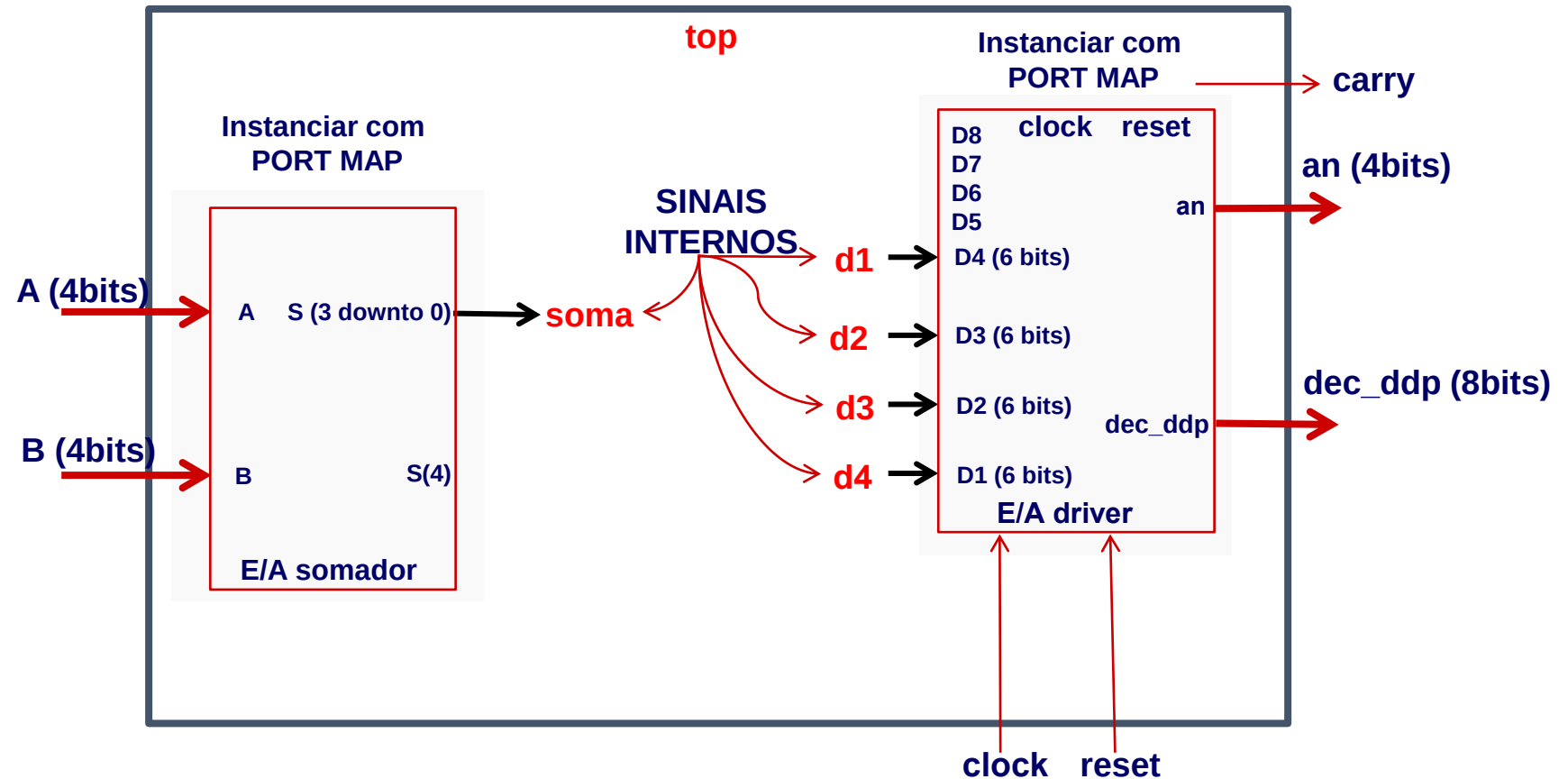
PROJETO 2

1. Coloque no projeto o fonte do somador e do acionador (*driver*) do mostrador(disponível [aqui](#))
2. Adicione o arquivo `.xdc` genérico e configure ele para que reflita a relação entre a entidade do módulo VHDL de mais alto nível hierárquico (*top*) e os pinos do FPGA conectados a dispositivos de entrada ou saída, conforme o desenho abaixo, à direita



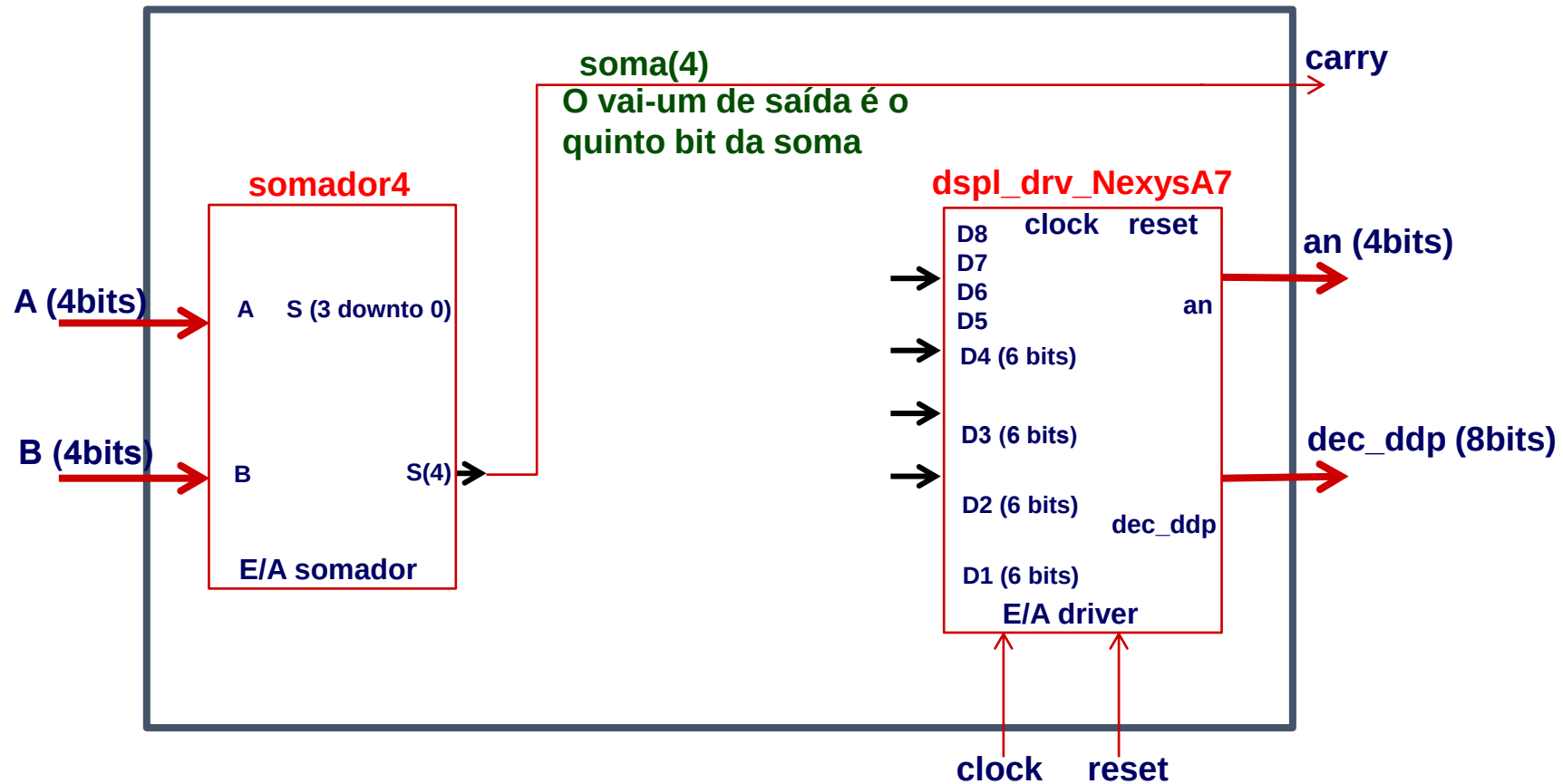
PROJETO 2

3. Crie um arquivo VHDL que faz o papel de topo da hierarquia de projeto e que será denominado `top.vhd`. Para inserir neste arquivo os 2 comandos `port map` (que instanciam o somador e o driver do mostrador), alguns sinais internos precisam ser criados, veja dicas abaixo



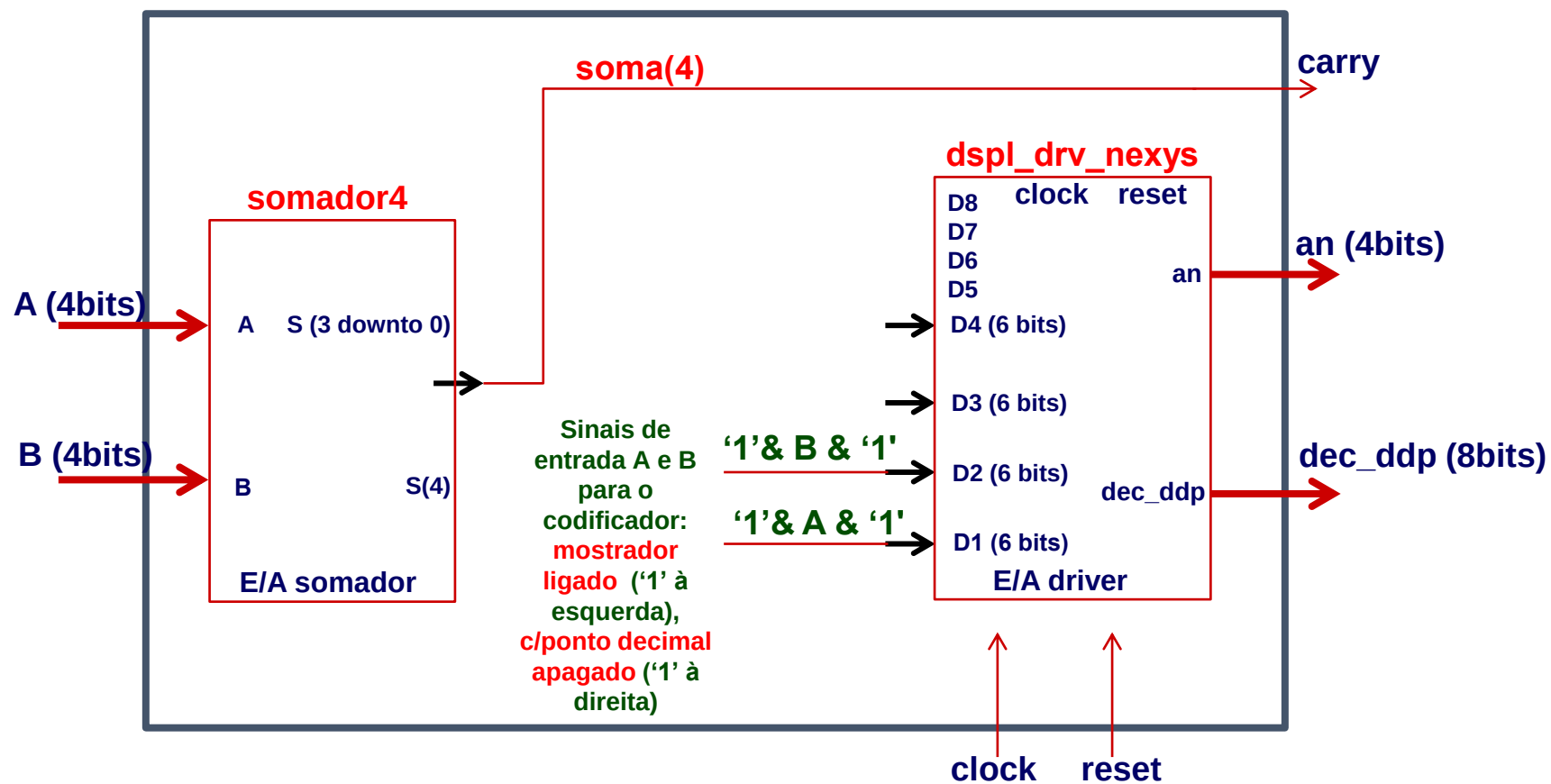
PROJETO 2

4. Ligue os sinais internos



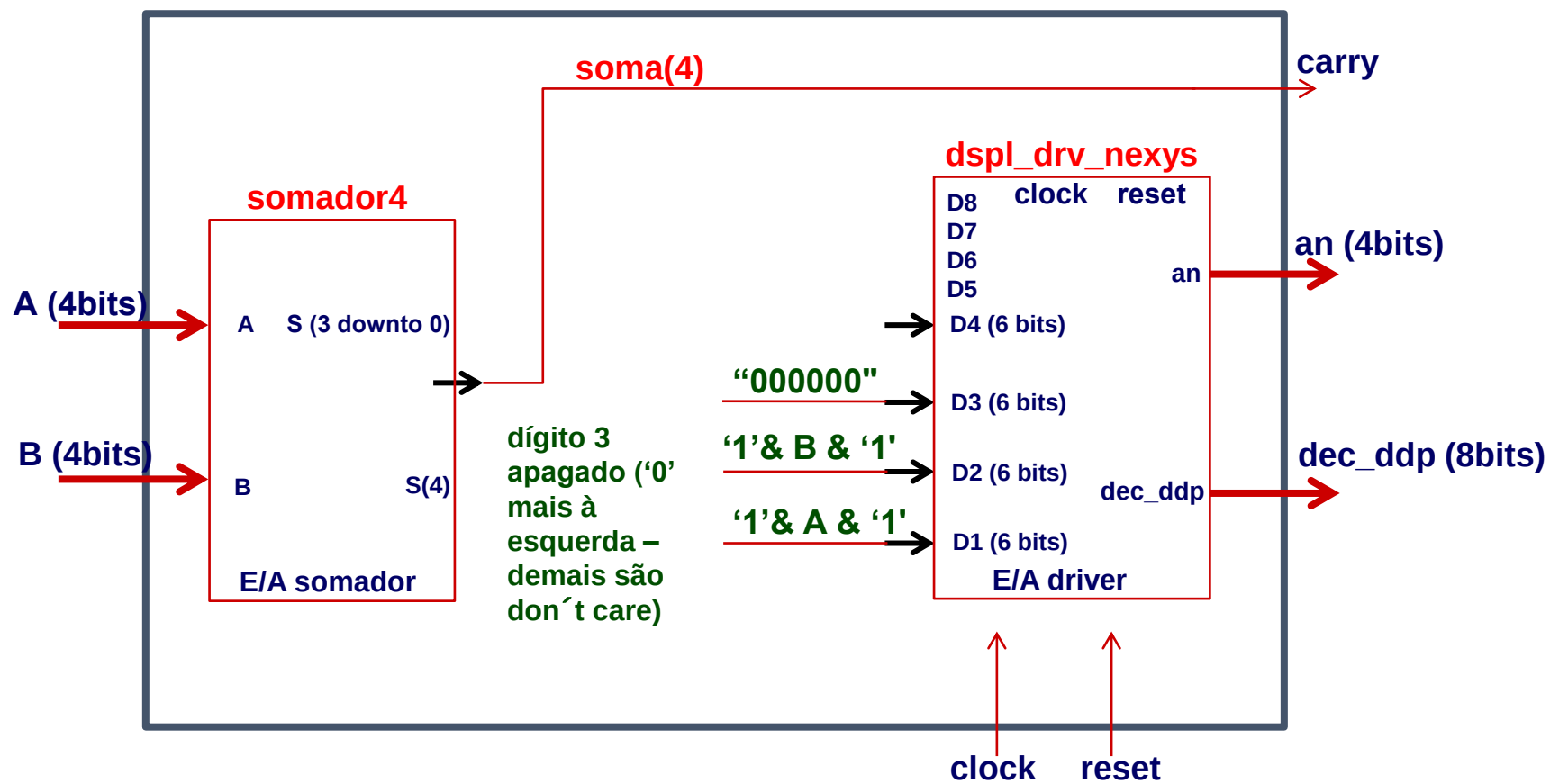
PROJETO 2

4. Ligue os sinais internos



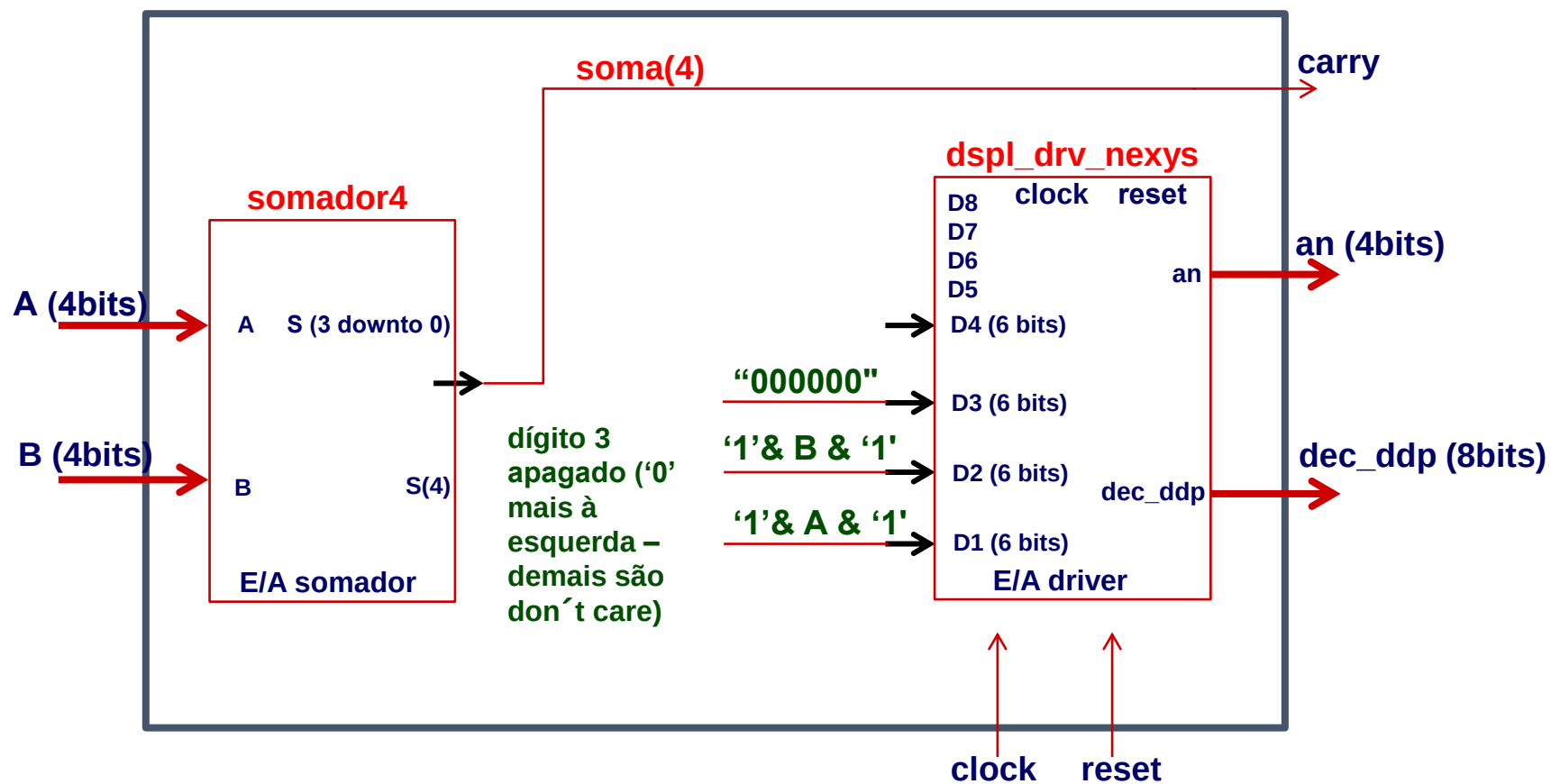
PROJETO 2

4. Ligue os sinais internos



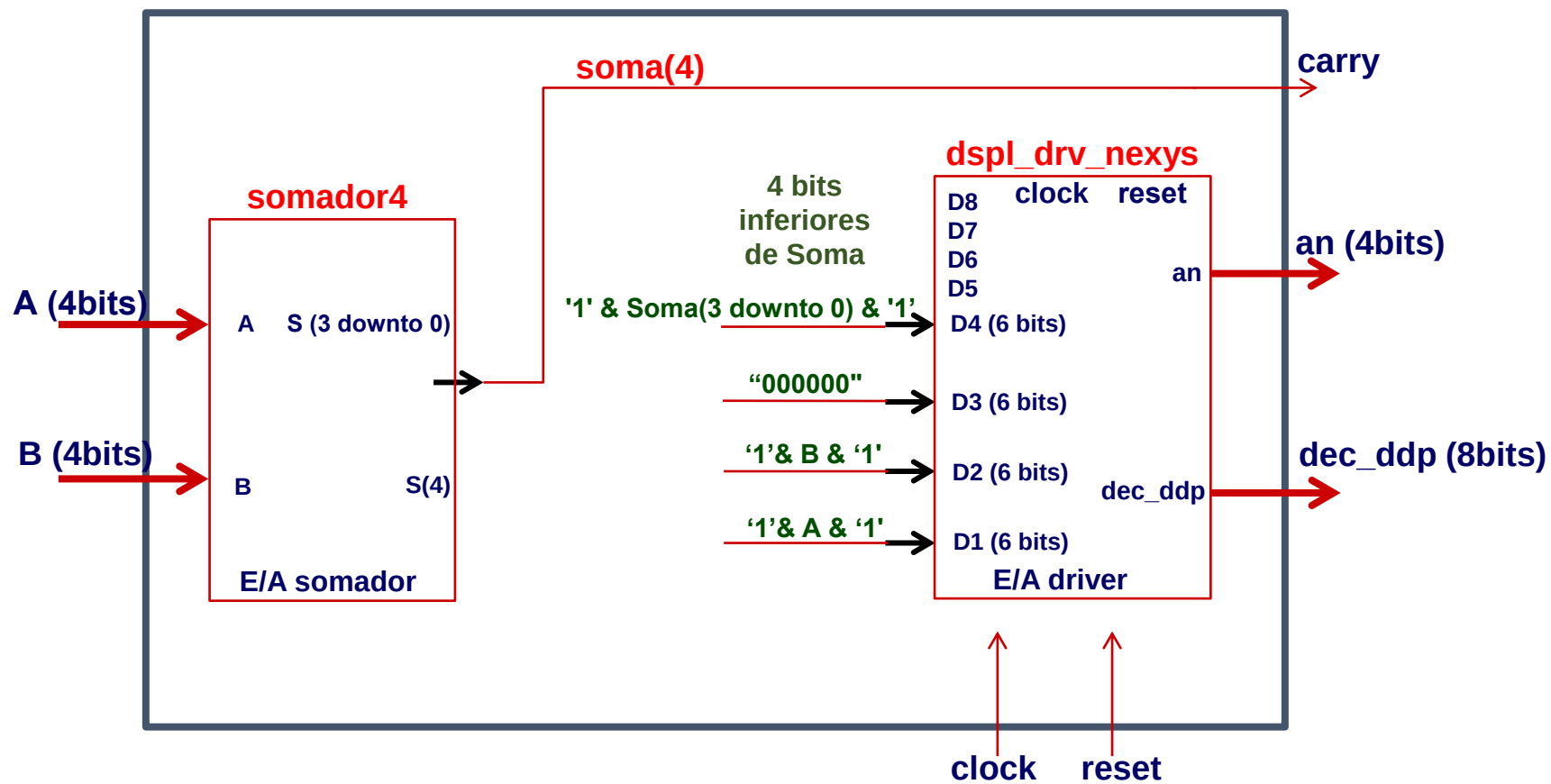
PROJETO 2

4. Ligue os sinais internos



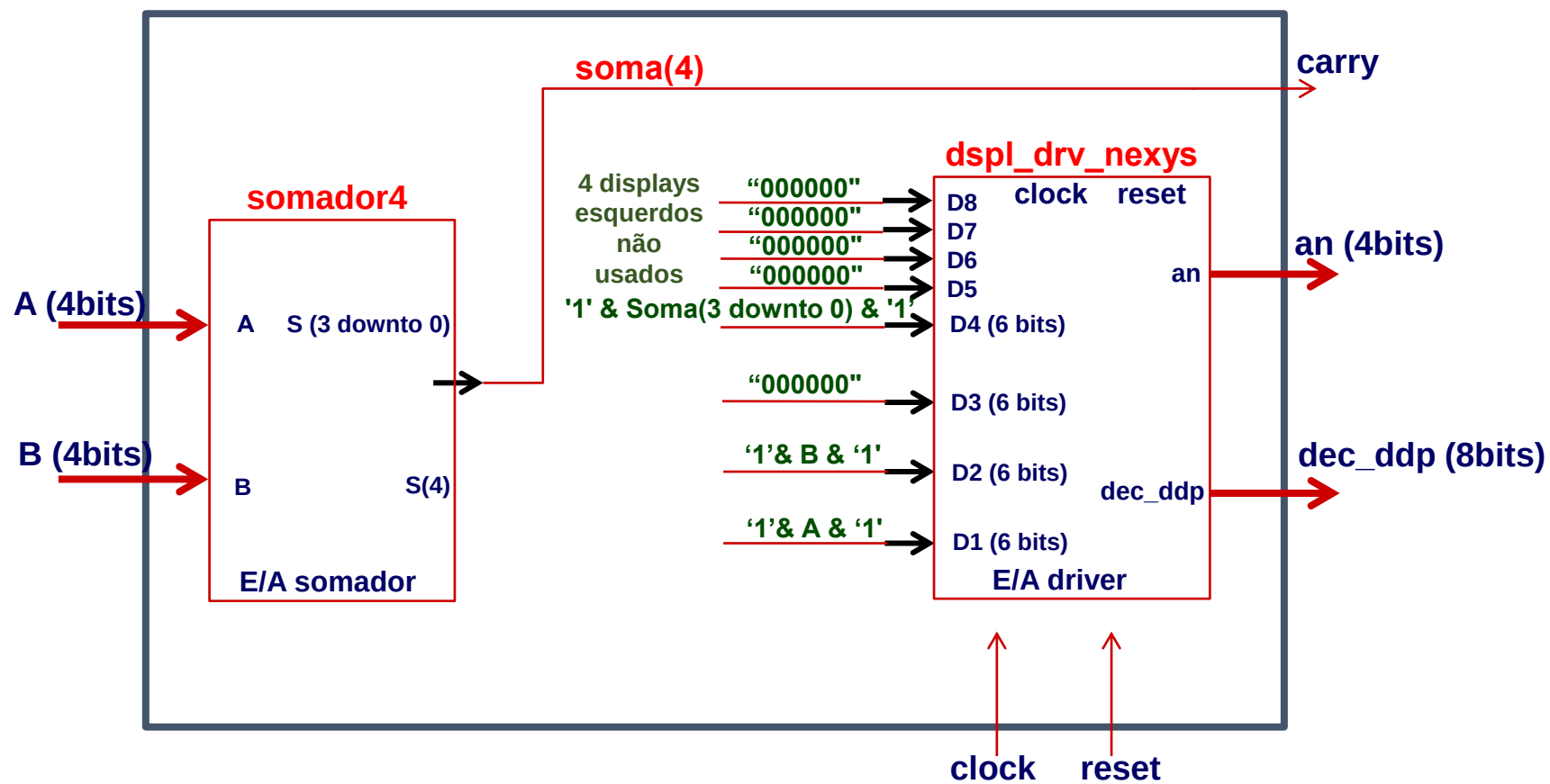
PROJETO 2

4. Ligue os sinais internos



PROJETO 2

4. Ligue os sinais internos



RESUMO DO TRABALHO T2

- A entrega do **Trabalho T2** consiste em fazer upload para o Moodle de um arquivo compactado contendo
 - ✓ Um relatório em formato PDF, descrevendo o Projeto 1 e o Projeto 2 (2 pontos)
 - ✓ Um diretório para o Projeto 1, contendo os seguintes arquivos (3 pontos)
 - VHDL do somador com vai-um
 - VHDL do testbench
 - arquivo **.xdc** do projeto
 - arquivo **.bit** do projeto
 - ✓ Um diretório para o Projeto 2, contendo os seguintes arquivos (5 pontos)
 - VHDL do top.vhd com as instâncias do somador e do acionador (*driver*) e outros arquivos fontes, caso tenham dividido o Projeto em vários fontes
 - arquivo **.xdc** do projeto
 - arquivo **.bit** do projeto